

Quiz-2

Total points 11/21 ?

Instructions to Students:

- Write your **FULL NAME IN CAPITAL LETTERS** at the top of the answer sheet.
- Write your **ROLL NUMBER IN CAPITAL LETTERS** clearly next to your name.
- Read each question carefully before answering.
- Attempt **all questions**. Each question may be **MCQ (one correct)** or **MSQ (multiple correct)**.
- For code-based questions, carefully analyze the given CUDA code snippets before answering.
- Mark your choice(s) clearly (e.g., Q1: a, Q2: b,c).
- **Do not open other tabs, notes, or external resources** while giving the exam.
- **Do not use mobile phones, calculators, or any external device** to assist with the quiz.
- **Do not leave your seat** unless permitted by the invigilator.
- **Do not share answers or discuss** with peers during the exam.

Important Notes

- Time allowed: **30 minutes only**.
- All responses must be submitted before the time limit – **late submissions will not be considered**.
- This is a **closed-book, closed-internet** exam.
- Each question carries equal marks unless otherwise specified.

The respondent's email (**cs25mtech11015@iith.ac.in**) was recorded on submission of this form.

0 of 0 points

Write your roll number in **CAPITAL** Letters *

CS25MTECH11015

Write your name in **CAPITAL** Letters *

LOCHAN SURYA TEJA NEELI

Below You can find your questions.

11 of 21 points



✗ Higher occupancy does not always improve performance because: * 0/1

- ☒ Thread divergence becomes more problematic with higher thread counts ✗
- ☒ Cache thrashing occurs when too many concurrent threads compete for cache lines ✓
- ☒ Register spilling to local memory can offset benefits of additional active warps ✓
- ☐ Memory-bound kernels may saturate bandwidth before reaching full occupancy

Correct answer

- ☒ Memory-bound kernels may saturate bandwidth before reaching full occupancy
- ☒ Cache thrashing occurs when too many concurrent threads compete for cache lines
- ☒ Register spilling to local memory can offset benefits of additional active warps

✓ What is/are performance inefficiency(s) in this reduction kernel? Assume *1/1
sdata is a shared memory variable.

```
// Reduction loop
for(int s = 1; s < blockDim.x; s *= 2) {
    if(tid % (2*s) == 0) {
        sdata[tid] += sdata[tid + s];
    }
    __syncthreads();
}
```

- ☐ No inefficiency
- ☒ Warp divergence due to the modulo operation ✓
- ☐ Unnecessary synchronization placement
- ☒ Bank conflicts in shared memory access ✓



✗ Do you think warp divergence is a common issue in prefixsum algorithms *0/1 on GPUs?

If Yes Why.

- ☒ PrefixSum requires synchronization across computation stages ✗
- ☐ Prefixsum algorithms inherently require atomic memory operations
- ☐ Warp Divergence is not at all an issue in prefixsum algorithms.
- ☒ Some threads conditionally participate based on their index in partial sums ✓

Correct answer

- ☒ Some threads conditionally participate based on their index in partial sums

✓ `__global__ void atomicKern(int *arr){`
`int idx = threadIdx.x + blockIdx.x*blockDim.x;`
`if(idx%2==0) atomicAdd(&arr[0],1);`
`else atomicAdd(&arr[1],1);`
`}` * 1/1

Configuration: blockDim.x=4, gridDim.x=2, arr initialized {0,0}

Which is/are possible output(s) of *arr* after execution of *atomicKern*?

- ☐ {8,0}
- ☐ {3,5}
- ☐ {2,6}
- ☒ {4,4} ✓



✗ Consider the following CUDA kernel that sums elements of an array *A* into **out*:

```
__global__ void sumKernel(float *A, float *out, int N) {  
    int idx = blockIdx.x * blockDim.x + threadIdx.x;  
    if (idx < N) {  
        float temp = 0.0f;  
        for (int i = 0; i < 1000; ++i) {  
            // Access pattern with stride  
            temp += A[idx + i * 32];  
        }  
        out[idx] = temp;  
    }  
}
```

Assume *blockDim.x* = 256, *gridDim.x* is sufficient to cover *N*, and *A* is a large array.

Which of the following statements are correct regarding memory access inefficiencies?

- ☒ The kernel has a constant stride of 32 ✗
- ☒ The kernel performs 1000 strided accesses per thread, causing significant cache misses and memory latency issues ✗
- ☒ Memory accesses are coalesced across threads. ✓
- ☐ The loop's access pattern causes scattered global memory reads, leading to poor memory coalescing.

Correct answer

- ☒ Memory accesses are coalesced across threads.



✗ Which of the following may cause low occupancy? *

0/1

- ☒ High register count per thread
- ☐ Excessive use of shared memory
- ☐ Proper alignment of global memory accesses
- ☐ Higher number of scalar private variables



Correct answer

- ☒ Excessive use of shared memory
- ☒ High register count per thread
- ☒ Higher number of scalar private variables



✗ `__global__ void kernel(int *x) {
 ++x[0];
}`

*0/1

`int main(){`

`//Assume x is allocated and initialized with zero`

`kernel<<<1, 2>>>(x);`

`}`

What of the following options is/are correct with respect to the value of x ?

- ☐ 1, with using atomics
- ☒ 2, with using atomics
- ☐ 2, without using atomics
- ☐ 1, without using atomics



Correct answer

- ☒ 2, without using atomics
- ☒ 1, without using atomics
- ☒ 2, with using atomics



✓ #define T 16 * 1/1

```

__shared__ float As[T][T], Bs[T][T];
int row = blockIdx.y*T + threadIdx.y;
int col = blockIdx.x*T + threadIdx.x;
float s = 0.0f;

for (int m = 0; m < N/T; ++m) {
    As[threadIdx.y][threadIdx.x] = A[row*N + m*T + threadIdx.x];
    Bs[threadIdx.y][threadIdx.x] = B[(m*T + threadIdx.y)*N + col];

    for (int k=0;k<T;++k)
        s += As[threadIdx.y][k] * Bs[k][threadIdx.x];
}
C[row*N + col] = s;

```

Which statements are true?

- ☒ A __syncthreads() is required after the two loads and before the inner k-loop. ✓
- ☐ Using volatile on As/Bs removes the need for __syncthreads().
- ☒ This code has a race: some threads may read shared memory before it is fully written. ✓
- ☐ Adding a single __syncthreads() at the top of the loop (before loads) fixes it.

✓ Memory coalescing efficiency drops significantly when 32 threads in a warp access addresses with which pattern? *1/1

- ☐ Random permutation of addresses within a 128-byte cache line
- ☐ Address sequence: 0, 4, 8, 12, ..., 124
- ☐ Reverse order access: 124, 120, 116, 112, ..., 0
- ☒ Addresses with stride equal to number of banks: 0, 32, 64, 96, ... ✓



✓ Which is true about CUDA shared memory? *

1/1

- ☐ Shared by all threads in a grid.
- ☐ Part of Global Memory
- ☐ Persist across kernel launches.
- ☒ Shared only within a block.



✓ Which statement(s) about tiled matrix multiplication using shared memory is/are correct? *

*1/1

- ☐ Larger tiles always improve performance
- ☐ Tiling is only useful for square matrices
- ☒ It reduces global memory accesses by reusing tiles
- ☒ Shared memory bank conflicts can degrade performance



✓ Why is synchronization necessary during reduction inside a block? *

1/1

- ☐ To increase the number of active warps by preventing idle threads
- ☒ To ensure data loaded by one thread is visible to others before accumulation
- ☐ To enforce compiler optimizations for instruction-level parallelism
- ☐ To automatically align data for coalesced memory access



✗ Consider this loop structure in a reduction kernel:

*

0/1

```
for (int offset = blockDim.x / 2; offset > 0; offset /= 2) {
    if (threadIdx.x < offset) {
        shared_data[threadIdx.x] += shared_data[threadIdx.x + offset];
    }
    __syncthreads();
}
```

Assume `shared_data` is a shared memory variable.

Which statements is/are correct about this implementation?

- ☐ The `__syncthreads()` call is unnecessary when `offset < 32` since threads within a warp execute in lockstep.
- ☐ Warp divergence decreases progressively as offset becomes smaller, improving performance in later iterations.
- ☒ This implementation will produce incorrect results if `blockDim.x` is not a power of 2. ✓
- ☒ The algorithm achieves optimal memory coalescing since consecutive threads access consecutive memory locations. ✗

Correct answer

- ☒ This implementation will produce incorrect results if `blockDim.x` is not a power of 2.

✓ Which best describe occupancy? *

1/1

- ☐ A way to ensure all threads finish at the same time
- ☒ A measure of active warps relative to hardware capacity ✓
- ☐ A performance metric that counts instructions per clock
- ☐ The measure of memory bandwidth utilized by global memory accesses



✓ In a prefixsum kernel that uses atomics for global synchronization, the **primary performance bottleneck** is typically: *1/1

- ☒ Serialization at atomic operations between thread blocks ✓
- ☐ Register spilling due to complex indexing calculations
- ☐ Shared memory bank conflicts during the local prefixsum phase
- ☐ Thread divergence in the tree-based prefixsum operations

✓ `__shared__ float tile[32][32];` * 1/1
`int tx = threadIdx.x, ty = threadIdx.y;`
`tile[ty][tx] = A[row*N + tx];`
`__syncthreads();`
`float v = tile[tx][ty]; // access pattern A`

Which of the following is/are true?

- ☐ For float (4B) on 32 banks this pattern will never result in bank conflicts.
- ☒ `tile[ty][tx]` store is coalesced; `tile[tx][ty]` read likely causes bank conflicts. ✓
- ☐ Reordering indices to `tile[tx][ty]` for load would fix global memory coalescing.
- ☒ Padding tile as `tile[32][33]` can reduce conflicts. ✓



✗ Which of the following issues could arise with this atomic code in shared *0/1 memory?

```
if (threadIdx.x == 0)
    shared_sum = 0;
```

```
__syncthreads();
```

```
atomicAdd(&shared_sum, threadIdx.x);
```

- ☐ There is no issue because __syncthreads() guarantees safe execution
- ☒ Race condition if some threads perform atomic adds before initialization ✗
- ☐ Compiler might optimize away writes unless memory fences are used
- ☐ Threads beyond the first warp might not synchronize properly

Correct answer

- ☒ There is no issue because __syncthreads() guarantees safe execution

✗ A kernel uses 40 registers per thread and 48KB of shared memory per *0/1 block with 256 threads per block. On a GPU with 65536 registers per SM and 96KB shared memory per SM, what limits occupancy?

- ☐ Hardware warp schedulers
- ☐ Shared memory usage
- ☒ Register usage ✗
- ☐ Thread block size

Correct answer

- ☒ Shared memory usage



✗ Which statement(s) is/are correct regarding loop unrolling in GPU programs?

*0/1

- ☒ Loop unrolling can reduce instruction cache misses
- ☐ Loop unrolling always reduces execution time
- ☒ Loop unrolling can increase instruction-level parallelism
- ☒ Unrolling loops may decrease occupancy sometimes

✗

✓

✓

Correct answer

- ☒ Loop unrolling can increase instruction-level parallelism
- ☒ Unrolling loops may decrease occupancy sometimes

✓ Padding a shared-memory 2D tile by +1 column (`float A_s[T][T+1];`) is primarily used to:

*1/1

- ☐ Increase shared memory throughput by aligning to cache lines.
- ☐ Increase occupancy.
- ☐ Reduce register pressure.
- ☒ Avoid bank conflicts.

✓



✗ In a hierarachical parallel reduction with 1024 elements, if each thread block handles 256 elements and uses shared memory, what is the theoretical minimum number of __syncthreads() calls required per block? *0/1

☐ 8

☐ 7

☒ 9

☐ 15

✗

Correct answer

☒ 8

This form was created inside of IIT Hyderabad. - [Contact form owner](#)

Google Forms



